

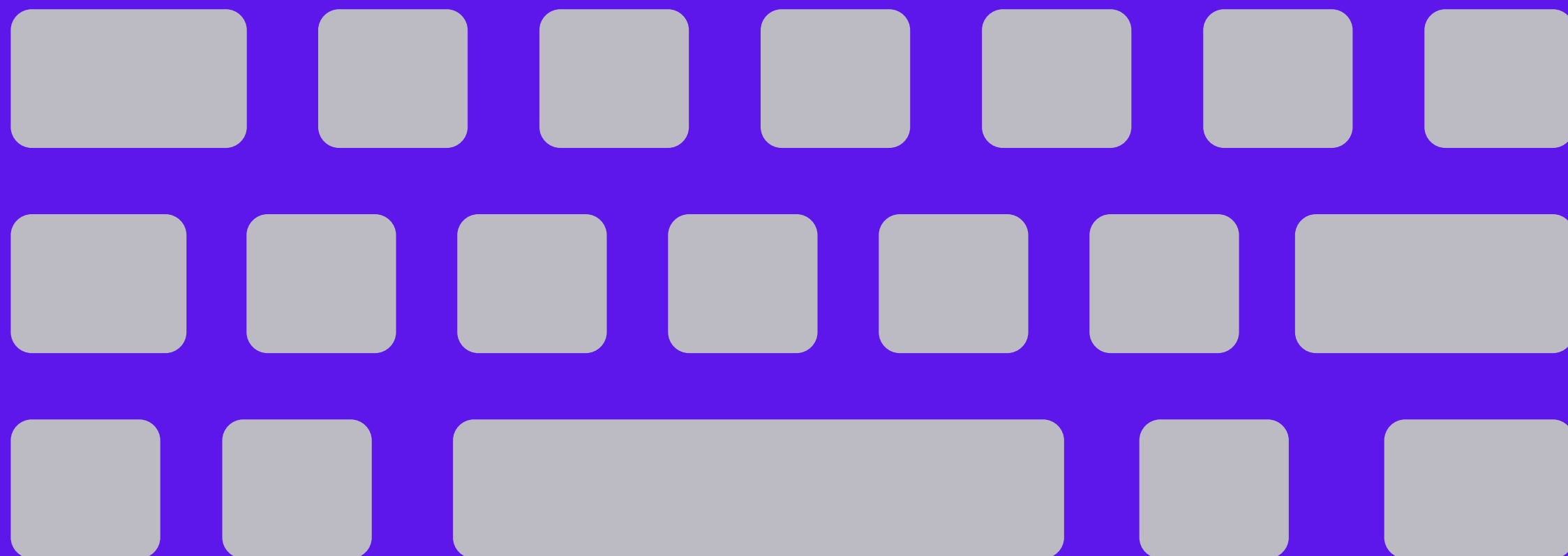


PROGRAMAÇÃO 101

Dia 1

- Introdução
- Diferentes Áreas de Especialização
- Linguagem de Baixo e Alto Nível
- Aplicações do Python
- Por que Aprender Python?
- Programação 101

Programação no Geral



Definição e Importância

A programação é a arte de criar instruções para um computador realizar tarefas específicas. É a linguagem que permite aos humanos interagirem com máquinas, transformando ideias em software funcional. A importância da programação transcende a simples criação de software, estendendo-se a diversos setores da sociedade.

Aplicações em Diversas Áreas

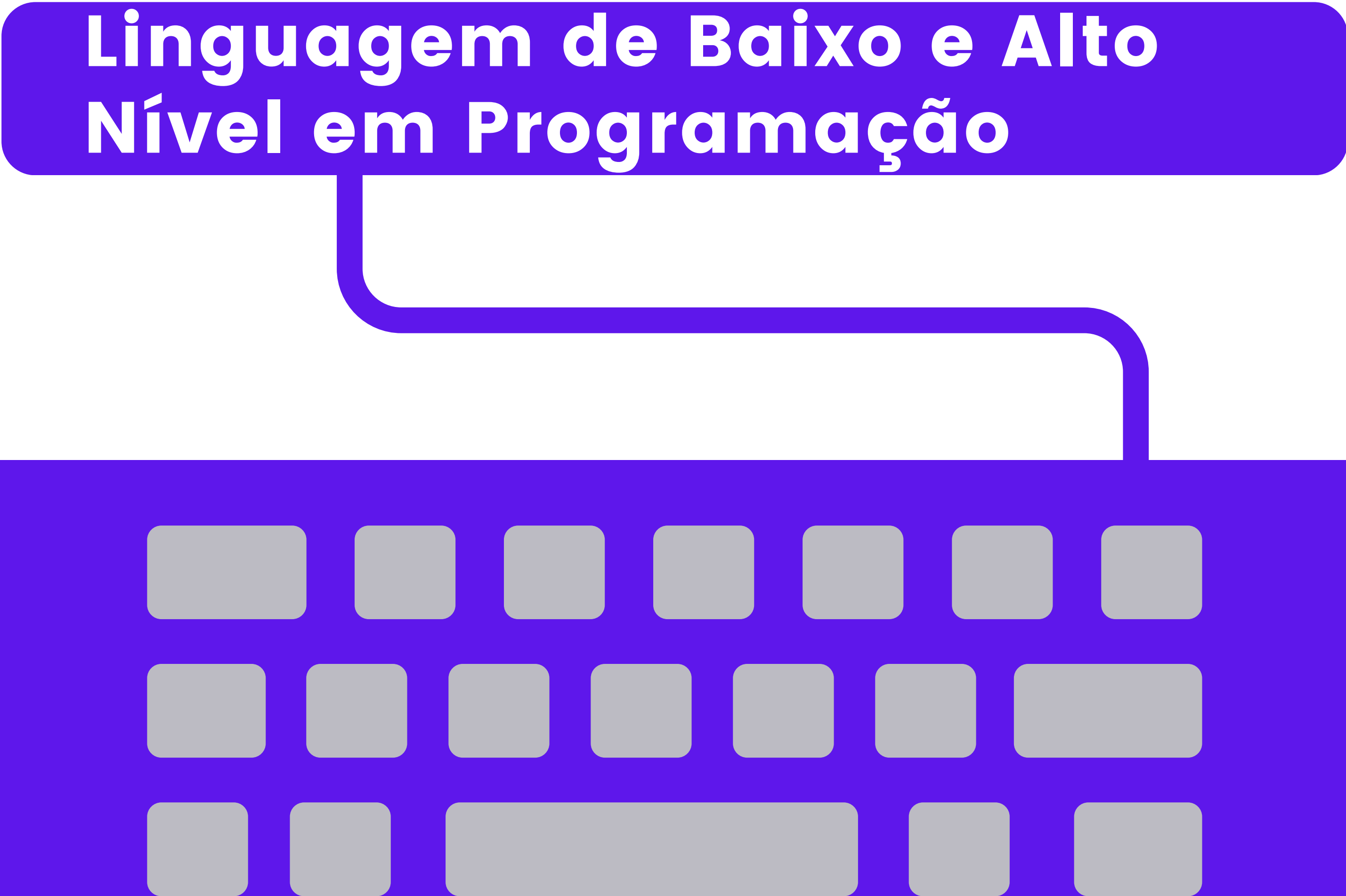
- Desenvolvimento Web:
 - Construção de sites interativos e dinâmicos.
 - Tecnologias como HTML, CSS, JavaScript e frameworks como Django, Flask (Python).
- Ciência de Dados e Inteligência Artificial:
 - Análise de dados para tomada de decisões.
 - Machine Learning e Deep Learning para criar modelos preditivos.
- Automação de Tarefas:
 - Automação de processos repetitivos para aumentar eficiência.
 - Uso de scripts para simplificar tarefas diárias.



Desenvolvimento de Software e Automação

- **Desenvolvimento de Software:**
 - Criação de aplicativos, sistemas operacionais e software de propósito geral.
 - Colaboração em equipes para criar produtos inovadores.
- **Automação:**
 - Controle de processos industriais.
 - Sistemas de automação residencial para otimizar o conforto e a segurança.

Linguagem de Baixo e Alto Nível em Programação



Tipos de linguagem

Linguagem de Baixo Nível:

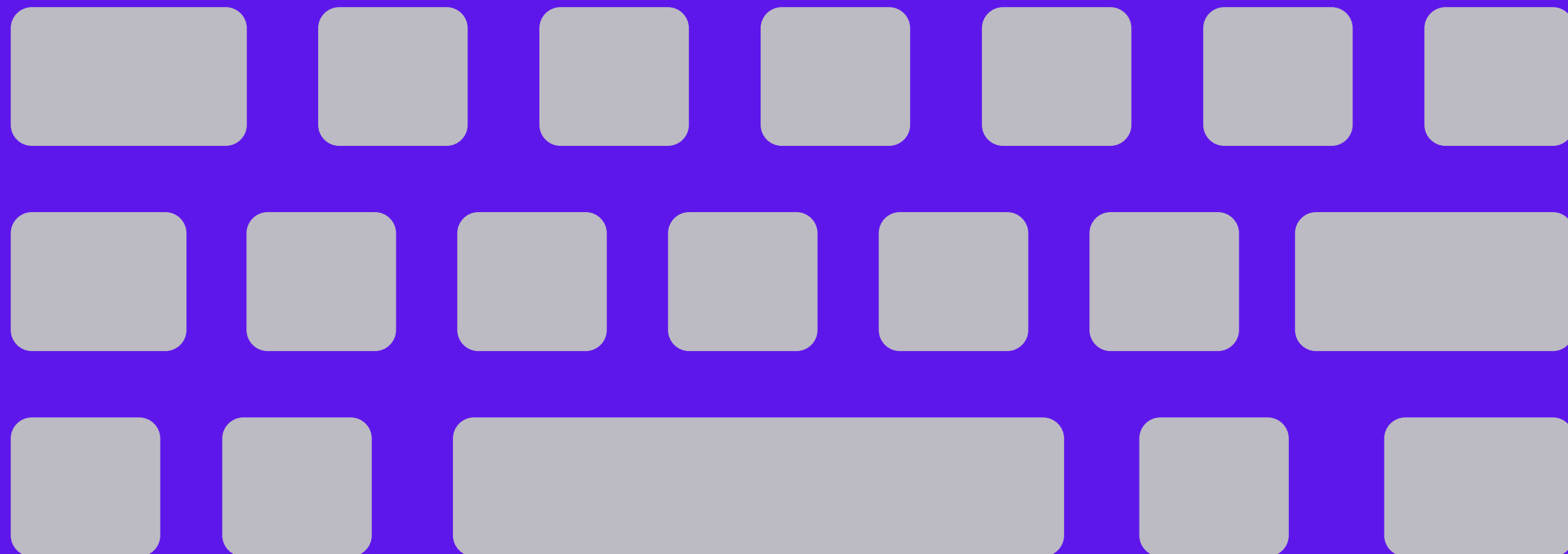
- Próxima ao Hardware
- Controle Fino
- Complexidade e Esforço
- Maior rapidez

Linguagem de Alto Nível:

- Abstração do Hardware
- Facilidade de Leitura e Escrita
- Automatização de Tarefas
- Portabilidade
- Demoram mais a executar

Aplicações do Python

```
graph TD; Title[Aplicações do Python] --- Grid[Grid of 21 placeholder boxes];
```

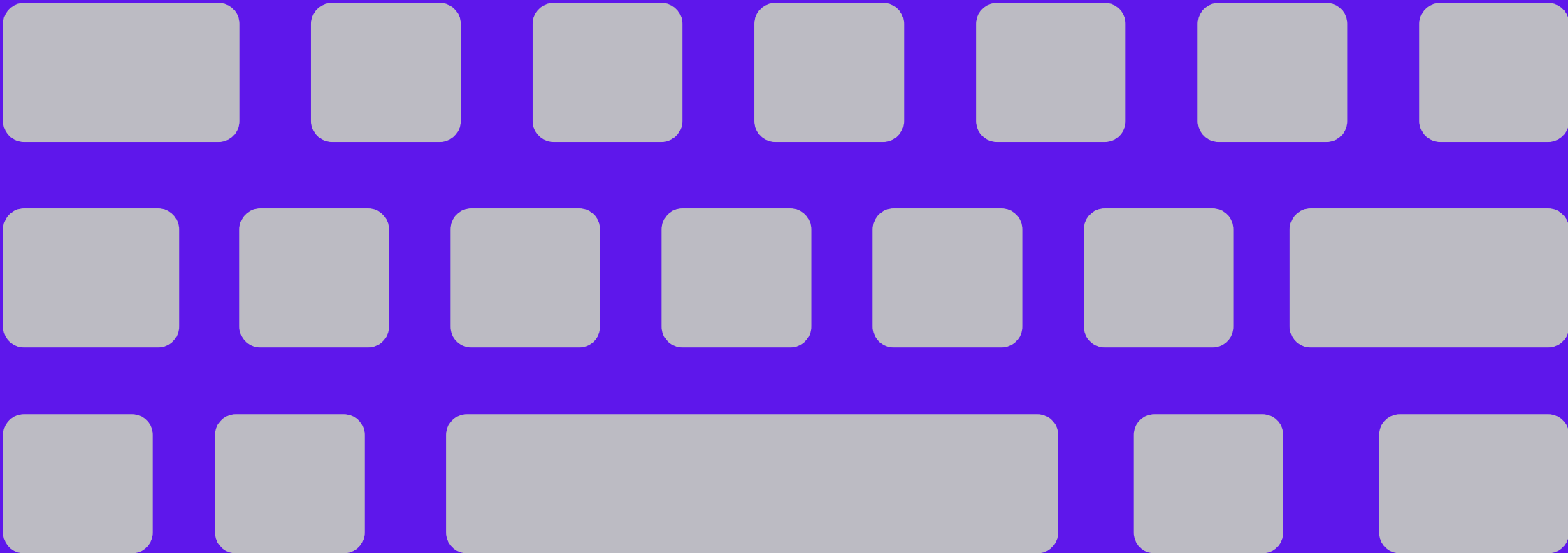
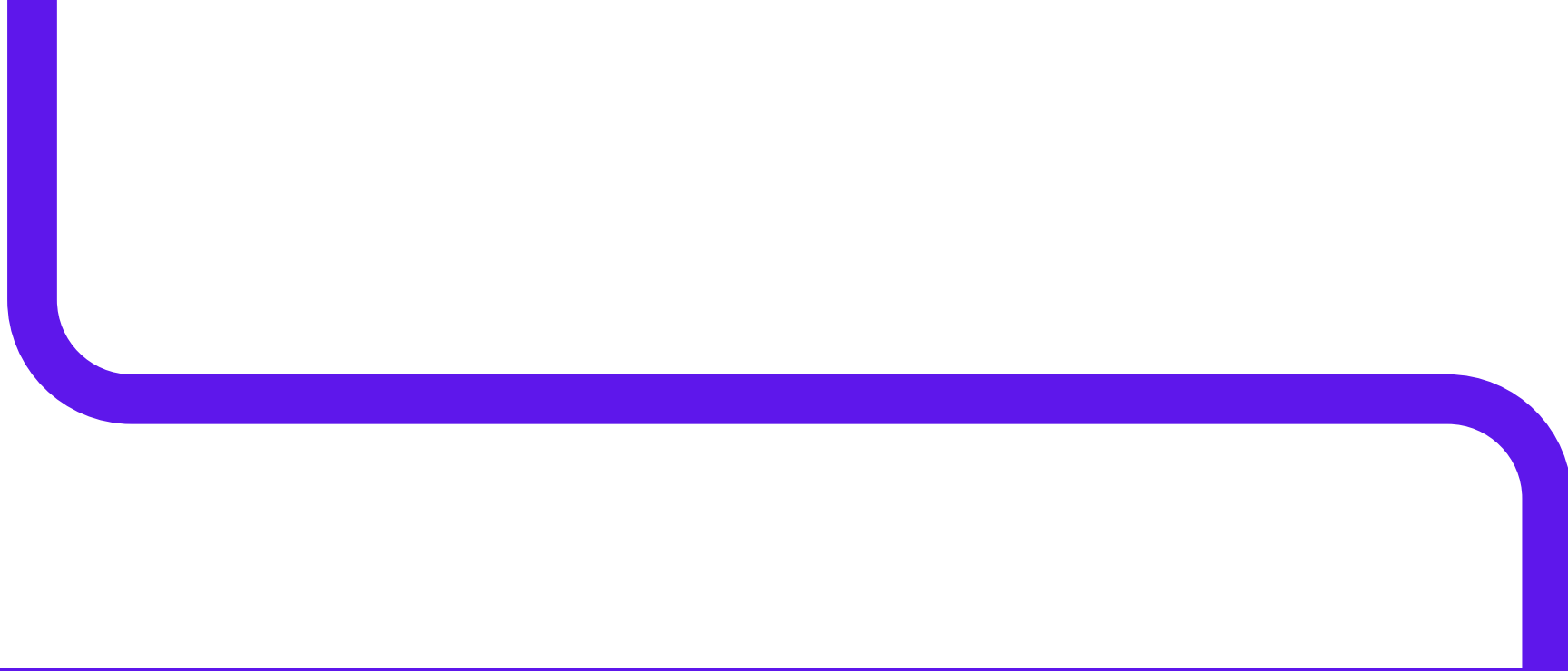


Aplicações do Python

- Scripting
- Desenvolvimento de Aplicações
- Análise de Dados e Machine Learning
- Automação de Tarefas
- ...



Por que Aprender Python?



Por que Aprender Python?

- 1. Sintaxe Clara e Legível:
 - Definição:
 - Python é conhecido por sua sintaxe clara e intuitiva, que se assemelha à linguagem natural.
 - Vantagens:
 - Facilita a leitura e escrita de código.
 - Reduz a probabilidade de erros.

Por que Aprender Python?

- 2. Grande Comunidade de Desenvolvedores:
 - Definição:
 - Python possui uma vasta comunidade global de desenvolvedores ativos.
 - Vantagens:
 - Suporte e colaboração com outros programadores.
 - Acesso a recursos e soluções prontas.

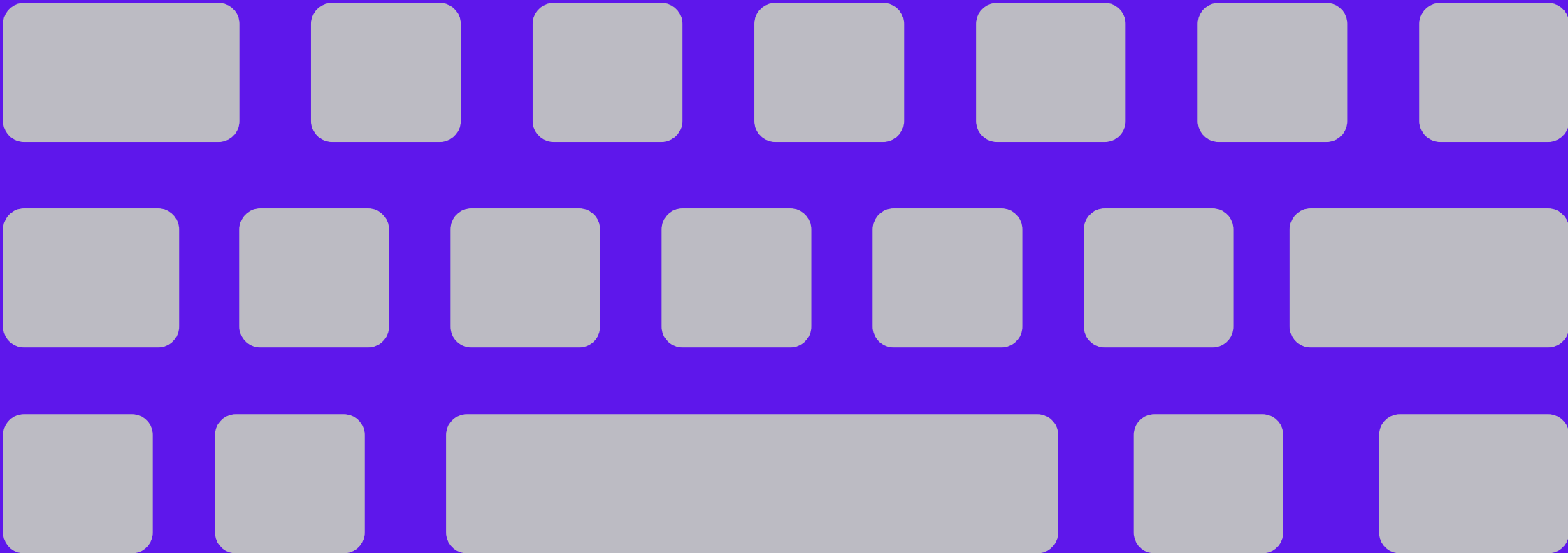
Por que Aprender Python?

- 3. Ampla Gama de Bibliotecas e Frameworks:
 - Definição:
 - Python oferece uma extensa variedade de bibliotecas e frameworks prontos para uso.
 - Vantagens:
 - Acelera o desenvolvimento de aplicações.
 - Cobertura de diversas áreas, como web, machine learning, ciência de dados.

Por que Aprender Python?

- 4. Versatilidade e Facilidade de Aprendizado:
 - Definição:
 - Python é versátil, adequado para uma ampla gama de aplicações, e é conhecido por ser amigável para iniciantes.
 - Vantagens:
 - Pode ser utilizado em diferentes domínios (web, automação, ciência de dados).
 - Facilidade de aprendizado, tornando-o ideal para iniciantes.

Programação 101

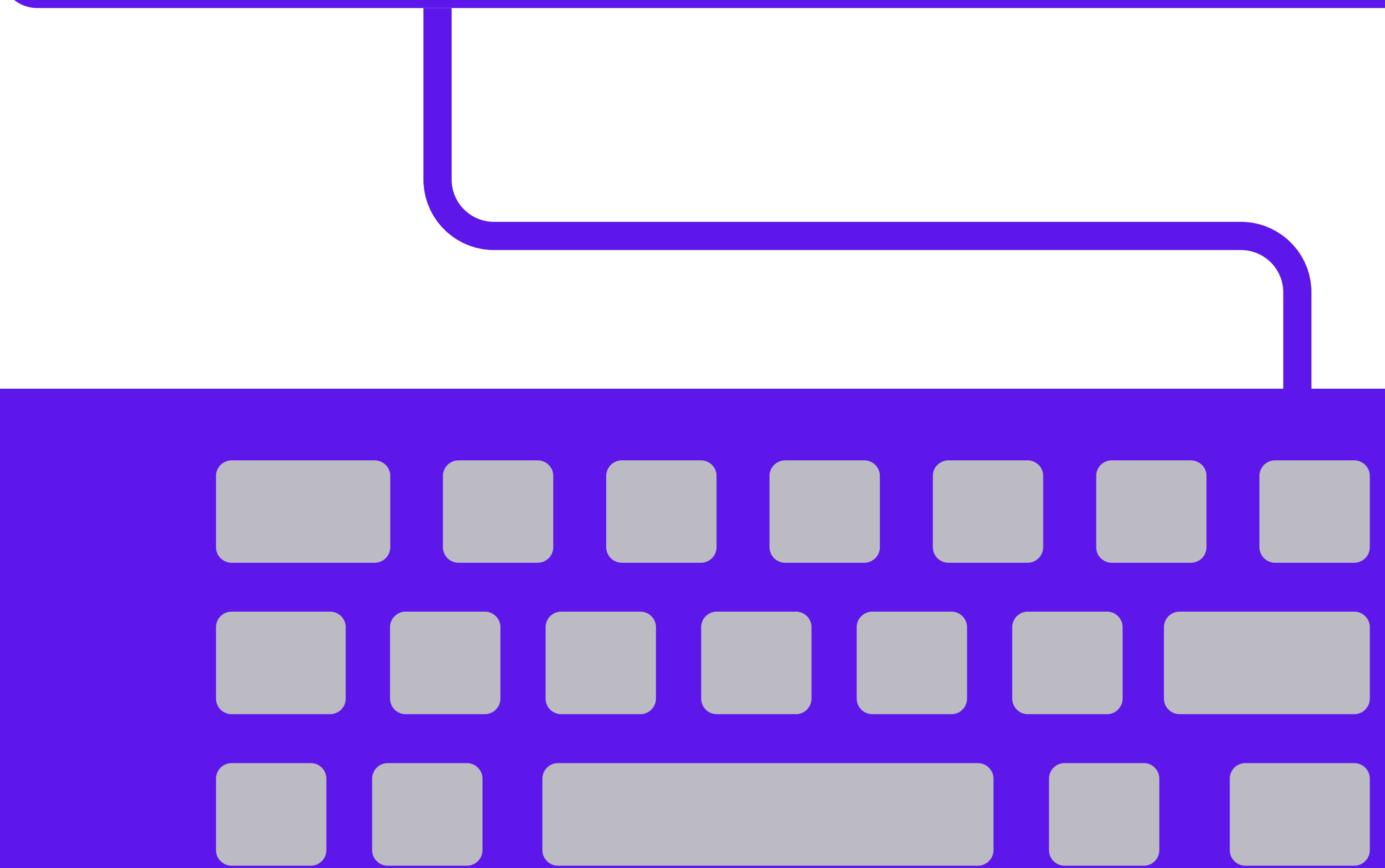


Por que Aprender Python?

- 5 dias de teoria e prática
- Uma visão geral sobre programação
- Aprender o básico de programação
- E muito mais!



Boas Práticas de Programação em Python



```
graph TD; Title[Boas Práticas de Programação em Python] --- Junction(( )); Junction --- Row1[ ]; Junction --- Row2[ ]; Junction --- Row3[ ]
```



A legibilidade do código é crucial para facilitar a manutenção, colaboração e compreensão. Em Python, seguimos as diretrizes estabelecidas no PEP 8 (Python Enhancement Proposal 8), que é o guia de estilo oficial para a linguagem. Algumas práticas importantes incluem:

Espaços em Branco:

- Use espaços em branco de maneira consistente para melhorar a legibilidade.
- Evite espaços em excesso, mas garanta que o código não fique apertado.

```
python

# Ruim
if(x==5):

# Bom
if x == 5:
```

A legibilidade do código é crucial para facilitar a manutenção, colaboração e compreensão. Em Python, seguimos as diretrizes estabelecidas no PEP 8 (Python Enhancement Proposal 8), que é o guia de estilo oficial para a linguagem. Algumas práticas importantes incluem:

Nomes de Variáveis Descritivos:

- Escolha nomes de variáveis que descrevam claramente sua finalidade.
- Prefira nomes legíveis em vez de abreviações obscuras.

```
# Ruim  
a = 10  
  
# Bom  
numero_de_estudantes = 10
```

A legibilidade do código é crucial para facilitar a manutenção, colaboração e compreensão. Em Python, seguimos as diretrizes estabelecidas no PEP 8 (Python Enhancement Proposal 8), que é o guia de estilo oficial para a linguagem. Algumas práticas importantes incluem:

Convenções de Nomenclatura:

- Siga as convenções de nomenclatura do PEP 8 para consistência.
- Por exemplo, use `snake_case` para nomes de variáveis e funções.

```
# Ruim
NomeDoUsuario = "John"

# Bom
nome_do_usuario = "John"
```


Comentários e documentação são elementos cruciais para comunicar a lógica do código e facilitar a compreensão para outros desenvolvedores ou para você mesmo no futuro.

Comentários para Explicar Lógica Complexa:

```
# Ruim - Lógica obscura
resultado = a * b + c / d

# Bom - Comentário explicativo
# Calcula o resultado usando a fórmula  $a * b + c / d$ 
resultado = a * b + c / d
```

Comentários e documentação são elementos cruciais para comunicar a lógica do código e facilitar a compreensão para outros desenvolvedores ou para você mesmo no futuro.

Manter Comentários Atualizados:

```
# Ruim - Comentário desatualizado  
# Próxima etapa: implementar a lógica de validação  
implementar_logica_validacao()  
  
# Bom - Comentário atualizado  
# Etapa concluída: implementada a lógica de validação  
implementar_logica_validacao()
```

Comentários e documentação são elementos cruciais para comunicar a lógica do código e facilitar a compreensão para outros desenvolvedores ou para você mesmo no futuro.

Documentação de Funções e Módulos:

```
# Ruim - Falta de documentação
def calcular_media(lista):
    return sum(lista) / len(lista)

# Bom - Documentação clara
def calcular_media(lista):
    """
    Calcula a média dos elementos em uma lista.

    Parameters:
    - lista (list): A lista de elementos numéricos.

    Returns:
    float: A média dos elementos na lista.
    """
    return sum(lista) / len(lista)
```


Evite Códigos Duplicados

Reutilização de Código

A duplicação de código pode levar a problemas de manutenção, aumentar a propensão a erros e dificultar a atualização do software.

```
# Ruim - Código duplicado
def calcular_area_retangulo(largura, altura):
    return largura * altura

def calcular_area_quadrado(lado):
    return lado * lado

# Bom - Reutilização de código
def calcular_area_retangulo(largura, altura):
    return largura * altura

def calcular_area_quadrado(lado):
    return calcular_area_retangulo(lado, lado)
```

Evite Códigos Duplicados

Uso de Funções e Classes

A duplicação de código pode levar a problemas de manutenção, aumentar a propensão a erros e dificultar a atualização do software.

```
# Ruim - Código repetido
resultado1 = a * b + c
resultado2 = a * b + c

# Bom - Uso de função
def calcular_resultado(a, b, c):
    return a * b + c

resultado1 = calcular_resultado(a, b, c)
resultado2 = calcular_resultado(a, b, c)
```

Evite Códigos Duplicados

A duplicação de código pode levar a problemas de manutenção, aumentar a propensão a erros e dificultar a atualização do software.

Facilitando a Manutenção

```
# Ruim - Código duplicado em diferentes partes do programa
if condicao:
    # Bloco A
    resultado = a * b + c
else:
    # Bloco B
    resultado = a * b + c

# Bom - Centralizando a lógica
resultado = a * b + c
if condicao:
    # Bloco A
    resultado += d
else:
    # Bloco B
    resultado -= d
```

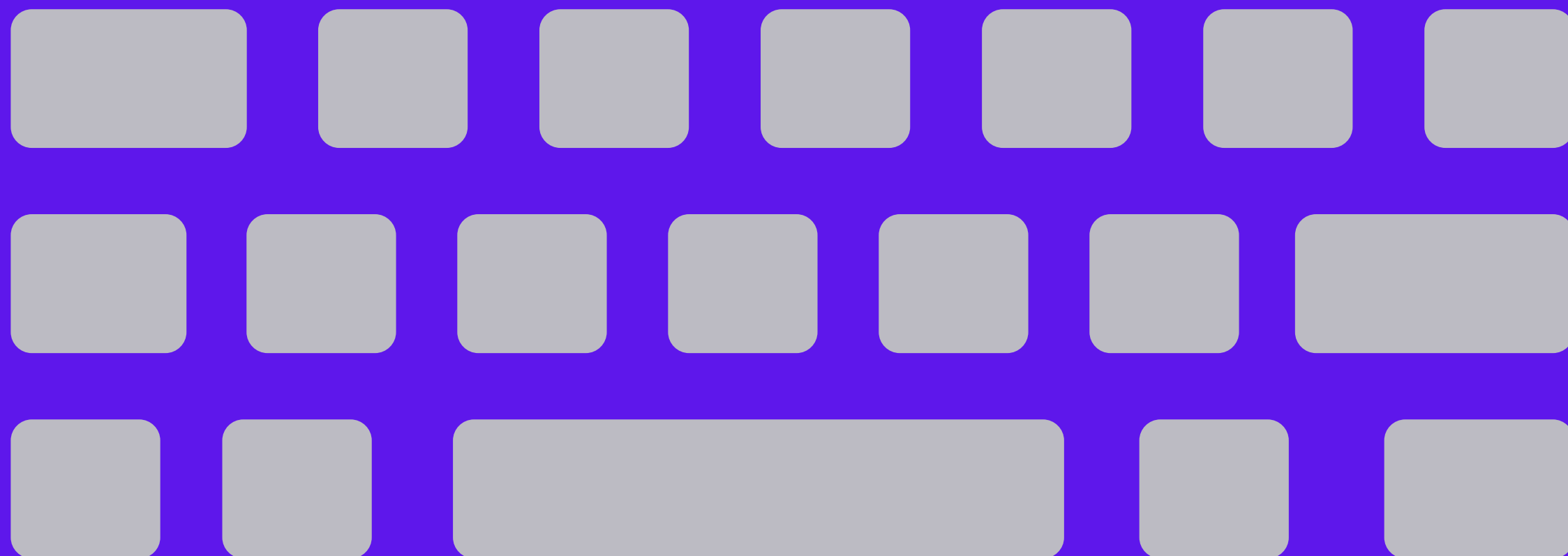
Outros tipos

- Estruturas de Controle:
 - Evite aninhar muitos níveis de indentação
 - Prefira o uso de elif em vez de múltiplos if
 - Mantenha linhas de código dentro de um limite razoável
- Tratamento de Exceções:
 - Use blocos try, except para lidar com exceções
 - Não utilize exceções para controlar fluxo normal do programa
 - Seja específico ao capturar exceções
- Uso Adequado de Listas e Dicionários:
 - Utilize compreensão de listas para operações simples
 - Evite listas e dicionários aninhados em excesso
 - Considere a eficiência ao manipular grandes conjuntos de dados
- Atualização de Versões e Bibliotecas:
 - Mantenha seu ambiente Python atualizado
 - Utilize virtual environments para isolar dependências
 - Verifique regularmente por atualizações de bibliotecas
- Testes e Debugging:
 - Escreva testes unitários para verificar a funcionalidade do código
 - Utilize ferramentas de debugging para identificar e corrigir problemas
 - Não subestime a importância dos testes
-

Dia 1

- Código
- Variáveis
- Convertendo tipos
- Testes

O que é um Código?

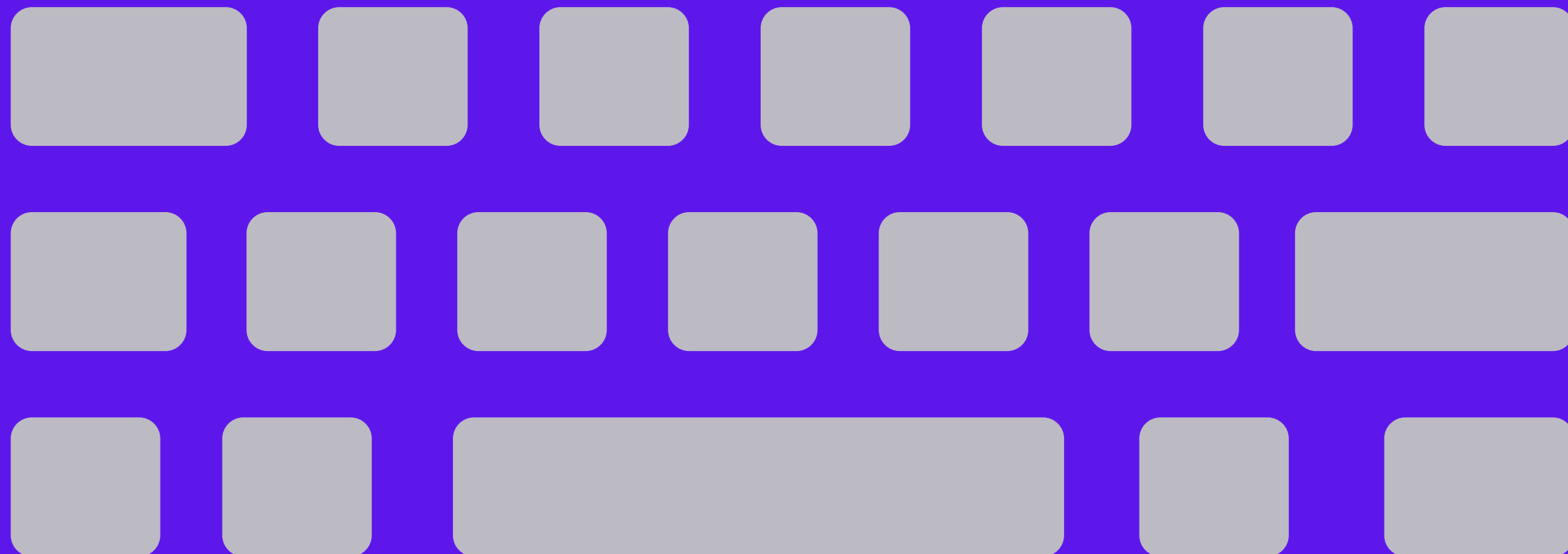


Definição de Código

Um código é um conjunto de instruções que são dadas ao computador. Essas instruções são escritas por meio de uma linguagem de programação, no caso o Python.

Quando executamos esse código, as instruções serão realizadas de maneira sequencial.

O que são Variáveis?

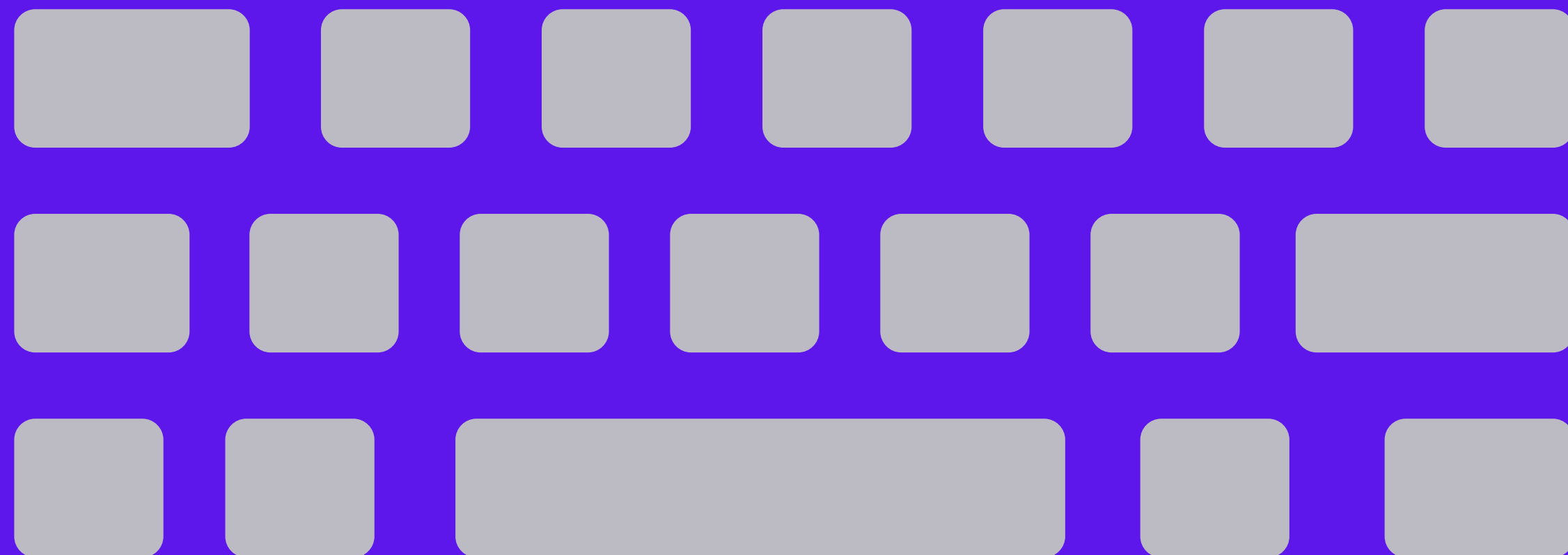


Definição e Importância

Variáveis são espaços de armazenamento que contêm dados referenciados por um nome.

```
nome = "John"  
idade = 25  
altura = 1.75
```

Tipos de Variáveis em Python:



Tipos de Variáveis

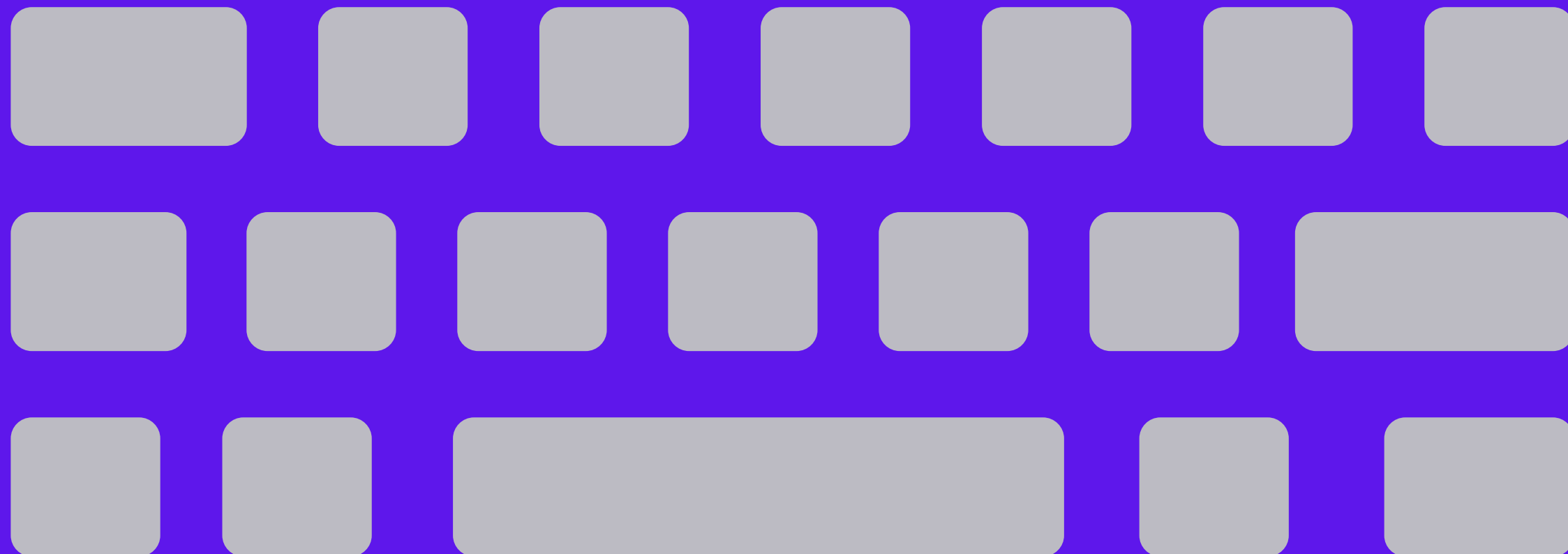
Tipos Básicos:

- int: Números inteiros.
- float: Números de ponto flutuante.
- str: Sequências de caracteres.
- bool: Valores lógicos (True/False).

```
numero_inteiro = 42  
numero_decimal = 3.14  
texto = "Olá, Python!"  
condicao = True
```

Métodos para Conferir o Tipo da Variável

```
graph TD; Title[Métodos para Conferir o Tipo da Variável] --- Grid[Grid of 21 boxes];
```



type()

Função type():

- Retorna o tipo da variável.

```
tipo_numero = type(42)
tipo_texto = type("Olá")
```

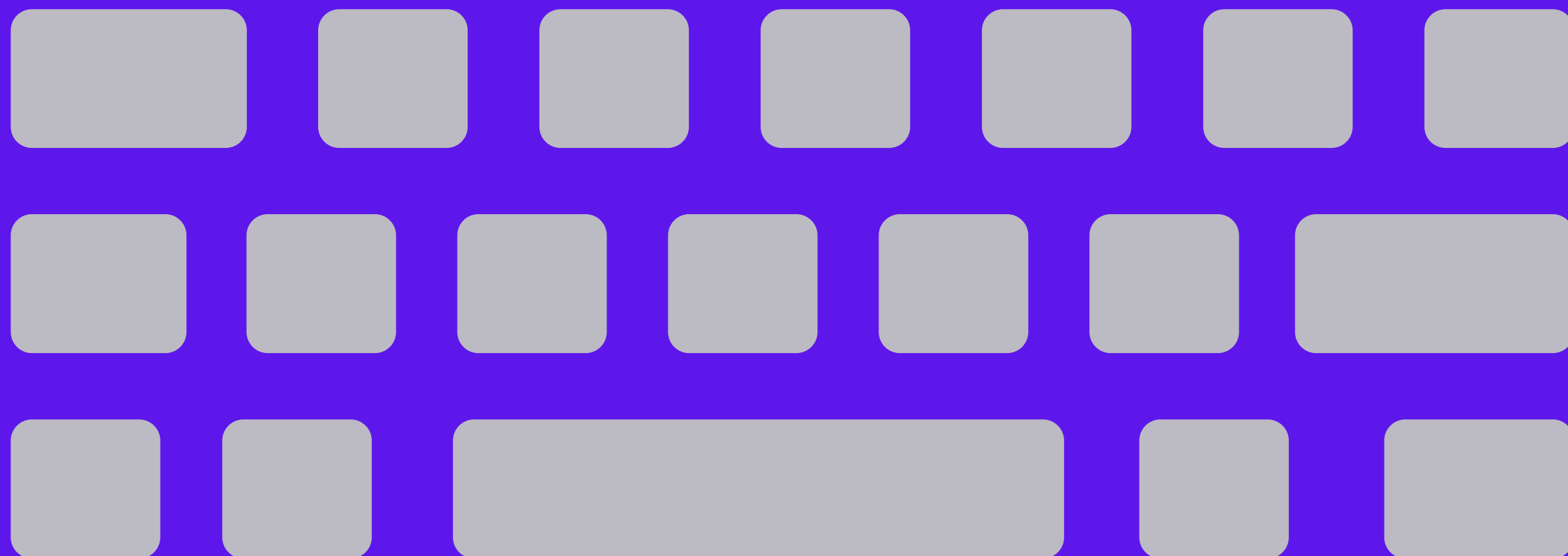

isinstance()

Método isinstance():

- Verifica se a variável é de um tipo específico.

```
resultado = isinstance(numero_inteiro, int)
```

Conversão de Variáveis

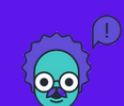


Conversão de Variáveis

Funções de Conversão:

- `int()`: Converte para inteiro.
- `float()`: Converte para ponto flutuante.
- `str()`: Converte para string.

```
numero_como_string = "42"  
numero_convertido = int(numero_como_string)
```



Dia 1

Operações em Python

Operadores em Python

Operação	Operador	Exemplo	Resultado
Soma	+	9 + 7	16
Subtração	-	9 - 7	2
Multiplicação	*	9 * 7	63
Divisão	/	5 / 2	2.5
Exponenciação	**	9 ** 2	81
Módulo	%	9 % 2	1

Realizando operações

- Utilizando os operadores em inteiros e decimais

```
resultado1 = 10 + 2  
resultado2 = 13.5 - 1.5
```

- Utilizando os operadores em variáveis

```
total = resultado1 * resultado2
```

Dia 1

Interagindo com o
Usuário em Python

Impressão de Valores

Função `print()`:

- Exibe valores na saída padrão (console).

```
idade = 25  
print("Idade:", idade)
```


Leitura de Mensagens

Função `input()`:

- Permite a leitura de mensagens inseridas pelo usuário.

```
nome = input("Digite seu nome: ")  
print("Olá, " + nome + "!")
```

Dia 1

Comentários de código

Comentários de código

Para criar comentários em seu código, ou seja, partes do código que o usuário vê mas o programa não lê, basta utilizar “#”

```
#Esse é meu código abaixo:  
  
print("Olá Mundo")  
  
# Ao usar a hashtag você comenta a linha inteira <3
```


Leitura de Arquivos

- Função `open()`:
 - Abre um arquivo para leitura.
- Método `read()`:
 - Lê o conteúdo do arquivo.

```
arquivo = open("exemplo.txt", "r")  
conteudo = arquivo.read()  
arquivo.close()
```

Leitura de Imagens

- Biblioteca PIL (Pillow):
 - Utilizada para trabalhar com imagens.
- Método `open()`:
 - Abre a imagem.
- Método `show()`:
 - Exibe a imagem.

```
from PIL import Image  
  
imagem = Image.open("exemplo.jpg")  
imagem.show()
```



PROGRAMAÇÃO 101